Toward Robust Machine Learning based Network Intrusion Detection Systems

Omar Anser

SnT — University of Luxembourg, Luxembourg

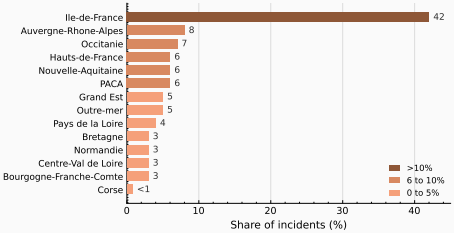
Jury

President:	Bernardetta Addis
Rapporteurs:	Nadjib Aït-Saadi, Francesco Bronzino
Examiner:	Grégory Blanc
Supervisors:	Isabelle Chrisment, Jérôme François

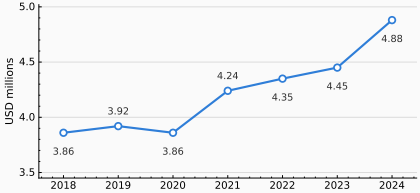
Introduction

Why this topic matters

- Modern societies and public services increasingly rely on computer networks $\longrightarrow$ Higher exposure to cyber incidents.
- ANSSI handled **4,386** security events in 2024 (**+15%** vs. 2023), including **1,361** confirmed incidents, i.e., nearly **4** per day.
- **$4.88M** average global cost of a data breach in 2024.
- **35%** of reported cyber incidents in the French health sector affected patient care.



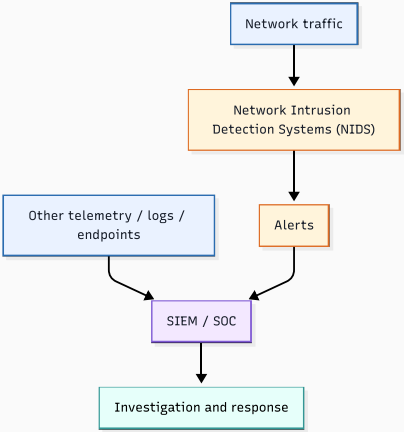
Source: **ANSSI, Panorama de la cybermenace 2024**



Source: **IBM Security, Cost of a Data Breach Report 2024**

Where NIDS fits in cyber defense

- Continuous monitoring to manage cyber risk
- SOCs and SIEM platforms centralize this monitoring
- Network traffic gives direct visibility into system communications
- NIDS turn traffic into alerts for investigation and response



⇒ **NIDS is one analytical layer inside the broader monitoring and response chain.**

Refs: NIST SP 800-86 (2006); Scarfone & Mell, Guide to Intrusion Detection and Prevention Systems (2007).

From signatures to ML-based NIDS

Rule-based NIDS

- Tools: **Snort**, **Suricata**, **Zeek**
- Detect only known attack patterns or protocol violations, leaving unknown ones undetected.
- Limited when payload content is encrypted

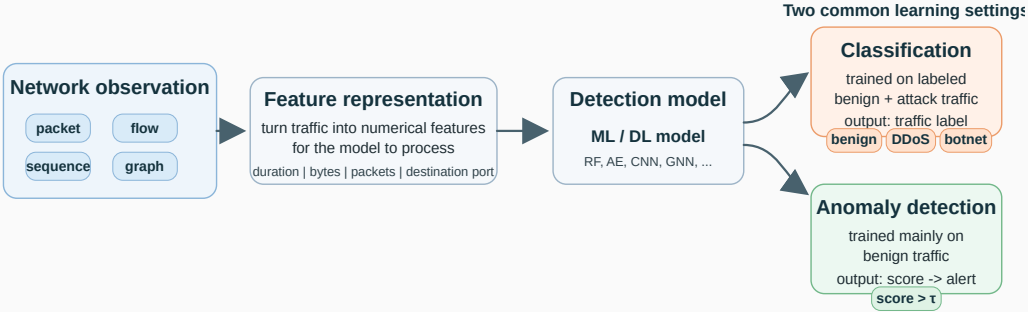
ML-based NIDS

- Example: **Random Forest (RF) on flows**
- Learn traffic patterns from historical data to generalize beyond exact rules.
- Can still detect suspicious behavior in encrypted traffic

Operational reality: mostly hybrid. For example, a SOC may use **Suricata/Zeek** for known alerts and **ML on flows** to flag suspicious HTTPS command-and-control behavior.

Refs: Snort [Roesch, 1999]; Zeek [Paxson, 1999]; Suricata (OISF); ML for NIDS [Sommer & Paxson, 2010]; encrypted traffic [Anderson & McGrew, 2016]; SANS Detection & Response Survey (2024).

ML-based NIDS as a prediction task

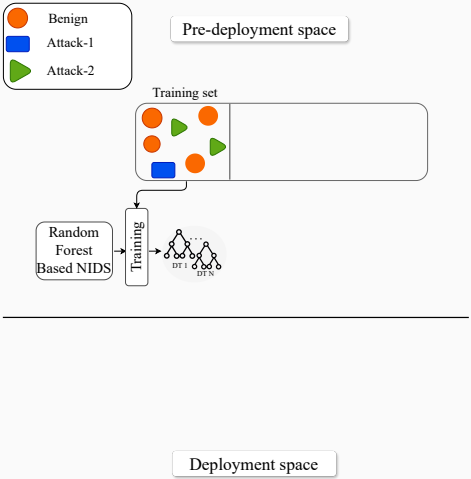


⇒ This common ML setting leads to the standard train → validate → test → deploy workflow.

Refs: Sommer & Paxson (2010); Buczak & Guven (2016).

ML-based NIDS Workflow: Train → Validate → Test → Deploy

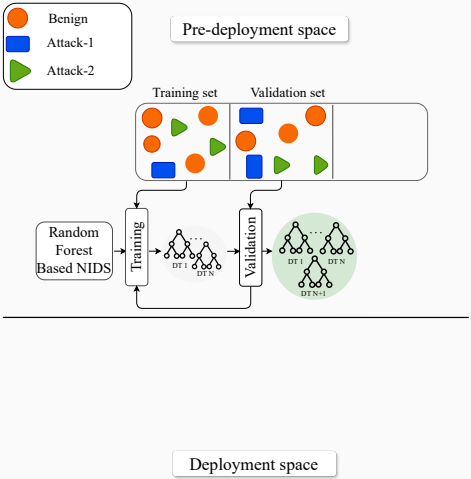
- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.



ML-based NIDS Workflow: Train → Validate → Test → Deploy

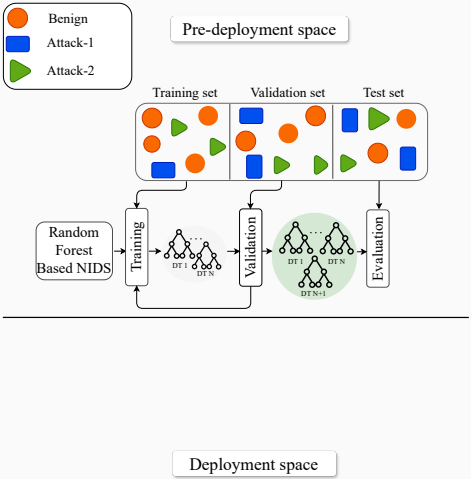
- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters** and, in anomaly detection, the **threshold**).

Hyperparameters: settings chosen before training that influence model behavior.
Example: In RF, too few trees can give unstable predictions, while too many can increase computation and overfit the training data.



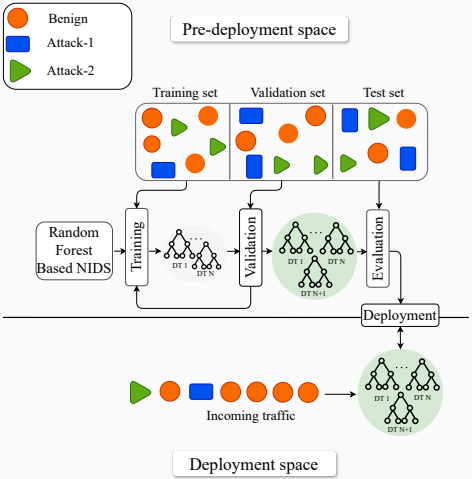
ML-based NIDS Workflow: Train → Validate → Test → Deploy

- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters** and, in anomaly detection, the **threshold**).
- A final **test** set, $\mathcal{D}_{\text{test}}$, measures performance on previously unseen data.



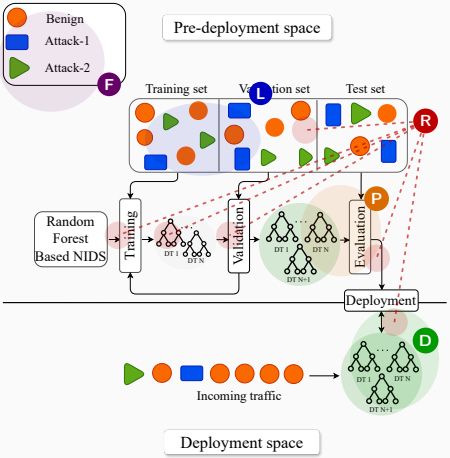
ML-based NIDS Workflow: Train → Validate → Test → Deploy

- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters** and, in anomaly detection, the **threshold**).
- A final **test** set, $\mathcal{D}_{\text{test}}$, measures performance on previously unseen data.
- The selected model is then **deployed** to monitor live traffic.



How ML has been used to improve NIDS

- **F** Richer traffic representations ⟶ Flow-MAE, NetVigil
  - e.g., packet, flow, sequence, graph
- **L** Better use of limited labels ⟶ Active-SVDD, INSOMNIA
  - e.g., active learning, semi-supervised learning, and pseudo-labeling
- **P** Better detection performance ⟶ BotMiner, Kitsune
  - e.g., DDoS, botnet, web attacks
- **D** More practical deployment ⟶ AOC-IDS, ENIDrift
  - e.g., lightweight, online, real-time detection
- **R** Better robustness in practice ⟶ CADE, OWAD
  - e.g., adaptability to evolving traffic, resilience to adversarial attacks, and more challenging evaluation



Refs: BotMiner [Gu et al., 2008]; Kitsune [Mirsky et al., 2018]; Flow-MAE [Hang et al., 2023]; NetVigil [Hsieh et al., 2024]; AOC-IDS [Zhao et al., 2024]; ENIDrift [Wang, 2022]; Active-SVDD [Gornitz et al., 2009]; INSOMNIA [Pierazzi et al., 2021]; CADE [Yang et al., 2021]; OWAD [Hu et al., 2023].

Focus of this thesis: robustness in ML-based NIDS

Robustness in this thesis

- Good performance on one fixed benchmark split is not sufficient to claim robustness.
- Before deployment, robustness depends on evaluation conditions that reveal detector weaknesses.
- After deployment, robustness means remaining effective when traffic and attack patterns evolve.

$\Rightarrow$ **ML-based NIDS robustness is studied both before deployment, through the evaluation process, and after deployment, through adaptability to evolving traffic.**

Roadmap of the presentation

Part I Evaluation

- Anser et al., *"TATA: Benchmark NIDS Test Sets Assessment and Targeted Augmentation,"*
  ESORICS 2025 (*main track*).
- Anser et al., *"An Empirical Study on Configuration Robustness of Unsupervised IDS,"*
  NOMS 2026 (*main track, to appear*).

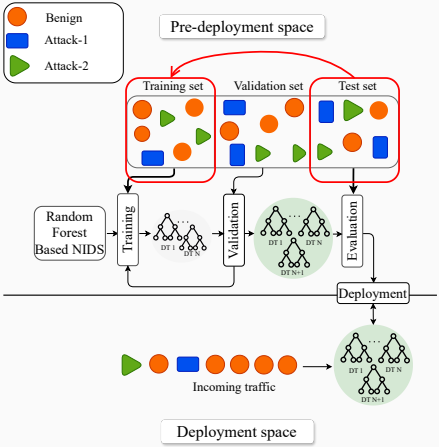
Part II Adaptation

- Anser et al., *"Auto-tuning of Hyper-parameters for Detecting Network Intrusions via Meta-learning,"*
  NOMS 2023 (*AnNet workshop*).
- Anser et al., *"Automated Machine Learning Configuration to Learn Intrusion Detectors on Attack-Free Datasets,"*
  LCN 2024 (*short paper*).

Part I: TATA

Introduction

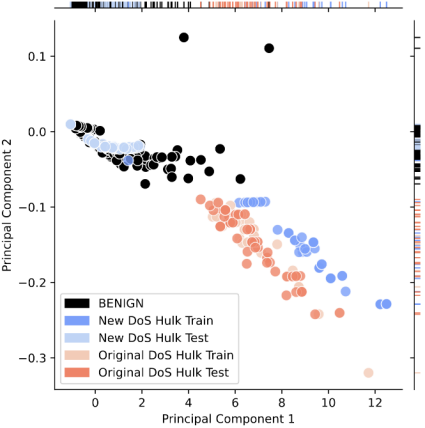
- A robust evaluation should expose challenging cases, including corner cases.
- Pre-deployment evaluation mainly relies on the test set $\mathcal{D}_{\text{test}}$.
- The key question is whether $\mathcal{D}_{\text{test}}$ really stresses the ML-based NIDS.
- The detector is constructed from $\mathcal{D}_{\text{train}} \rightarrow \mathcal{D}_{\text{test}}$ must be assessed relative to $\mathcal{D}_{\text{train}}$.



$\Rightarrow$ **This part focuses on $\mathcal{D}_{\text{test}}$ and its relation to $\mathcal{D}_{\text{train}}$.**

Benchmark NIDS datasets

- A single source dataset partitioned into $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$.
- Reported evaluation accuracies $>$ **90%** in published works on public benchmarks (IDS2017, UNSW-NB15...) $\rightarrow$ solved problem or benchmark-induced illusions?
- A near-complete overlap of DoS-Hulk flows in IDS2017's $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$[1].



Overlap between $\mathcal{D}_{\text{test}}$ and $\mathcal{D}_{\text{train}}$ does not prevent overfitting $\rightarrow$ it can hide it.

[1] Flood et al., Bad Design Smells in Benchmark NIDS Datasets, IEEE EuroS&P 2024.

RQ2 — Targeted Augmentation

Given a **too-easy** initial $\mathcal{D}_{\text{test}}$, can we apply systematic augmentation to generate new, **realistic**, and **challenging** traffic that is specifically designed to expose the hidden weaknesses of the NIDS?

Research Questions

RQ1 — Test-set Assessment

How can we create a numerical score or a set of metrics to objectively measure the difficulty of a $\mathcal{D}_{\text{test}}$ for a NIDS?

RQ2 — Targeted Augmentation

Given a **too-easy** initial $\mathcal{D}_{\text{test}}$, can we apply systematic augmentation to generate new, **realistic**, and **challenging** traffic that is specifically designed to expose the hidden weaknesses of the NIDS?

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage			

Neuron coverage: Fraction of neurons whose activation exceeds a preset threshold at least once across the test set; higher → explore more of the model's internal logic.

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage			
Surprise coverage			

Surprise coverage: Measures how broadly the test set differs from what the model saw in training; higher → the test set includes highly surprising inputs.

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage			
Surprise coverage			
Mutation testing			

Mutation testing: Mutates the model and/or its training setup (e.g., weights, architecture, or hyperparameters), then checks if the test set can distinguish the mutant from the original model.

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage	✓	✗	Model access (🔓) + NN-only ( ) + Ignores train–test alignment (📏)
Surprise coverage	✓	✗	🔓 + + Proximity-only focus (🎯)
Mutation testing	✓	✓	🔓 + 📏 + Model-change sensitive (🛠) Retrain per dataset (🔄) + Synthetic data (🧪)
Manual (Flood et al., 2024)	✗	✓	Unguided augmentation (👆)

G1 Model-related gaps **G2** Shortcomings in assessing $\mathcal{D}_{\text{test}}$ → RQ1 **G3** Weaknesses in $\mathcal{D}_{\text{test}}$ augmentation workflows → RQ2

TATA (Test-set Assessment & Targeted Augmentation)

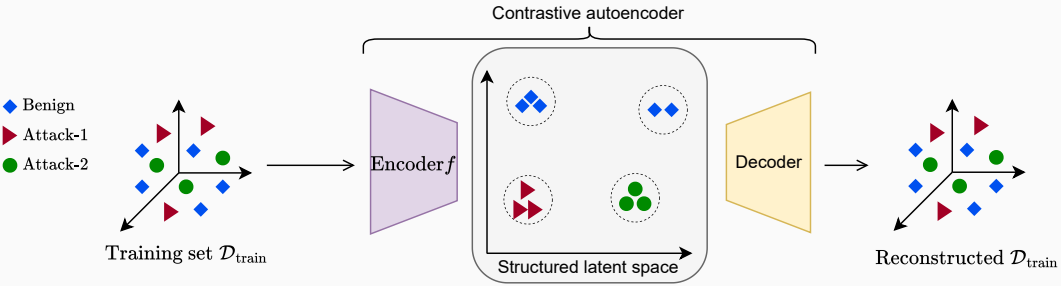
Goal: Close the gaps identified in related work and answer the research questions.

1. **Preliminary Phase** **G1**: + +
   Approximate $\mathcal{D}_{\text{train}}$'s decision boundary while being model agnostic

2. **Phase 1 – Assesses $\mathcal{D}_{\text{test}}$'s quality (RQ1)** **G2**: +
   Define three metrics (**Proximity**, **Diversity**, and **Scarcity**) to assess the **alignment** of $\mathcal{D}_{\text{test}}$ with respect to $\mathcal{D}_{\text{train}}$'s distribution.

3. **Phase 2 – $\mathcal{D}_{\text{test}}$ Augmentation (RQ2)** **G3**: + +
   Reinforcement learning (RL) guides a testbed to generate **realistic traffic** that fills uncovered metric gaps, producing **a more challenging** $\mathcal{D}_{\text{test}}$.

Preliminary Phase: Approximating $\mathcal{D}_{\text{train}}$'s decision boundary



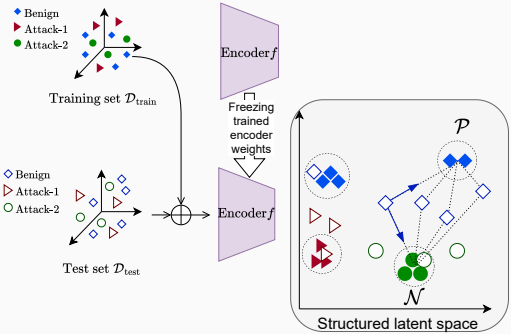
- Decision boundaries $\approx$ cluster borders in the latent space; these are captured in the encoder's weights.

Phase 1: Test-set Assessment

- Encode $\mathcal{D}_{\text{train}}$ & $\mathcal{D}_{\text{test}} \rightarrow \mathcal{Z}_{\text{test}}$ is located relative to $\mathcal{Z}_{\text{train}} \rightarrow$ we now work entirely in latent space $(\mathcal{Z}_{\text{train}}, \mathcal{Z}_{\text{test}})$.

- For each test embedding z, we identify:
  - $P(z)$: the nearest **same-label cluster** (*positive cluster*)
  - $N(z)$: the nearest **different-label cluster** (*negative cluster*)

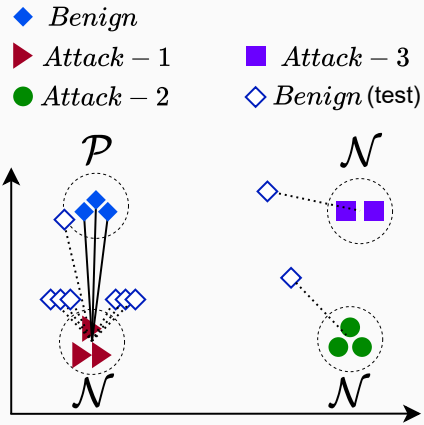
- Define proximity, diversity, and scarcity per $P(z)$; then aggreate.



Proximity $P \in [0, 1]$, higher = nearer to different-label boundaries

- Quantifies $\mathcal{Z}_{\text{test}}$ closeness to $N(z)$; high proximity $\Rightarrow$ closer to a different label decision boundary $\Rightarrow$ harder cases.

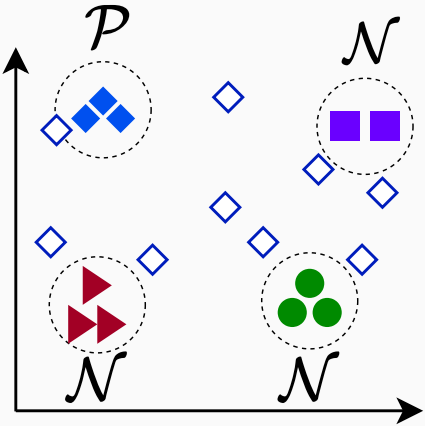
- P = **max** one-sided Kolmogorov–Smirnov statistic (over clusters): compares $\mathcal{Z}_{\text{test}}$ vs. $\mathcal{Z}_{\text{train}}$ distances to $N(z)$



Diversity $D \in [0, 1]$, higher = broader, less redundant coverage

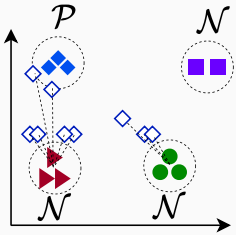
- Assesses the variety of $\mathcal{Z}_{\text{test}}$; higher diversity $\Rightarrow$ minimal redundancy $\Rightarrow$ broader coverage of the latent space.

- D = **Average** of normalized **Vendi Scores** (over clusters)

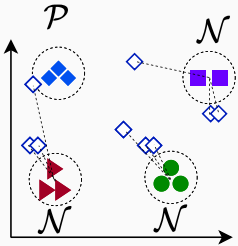


Scarcity $S \in [0, 1]$ higher = boundaries challenged more evenly

- Evaluates how uniformly $\mathcal{Z}_{\text{test}}$ are distributed among all potential negative clusters (i.e., decision boundaries).
- High scarcity $\Rightarrow$ a uniform spread $\Rightarrow$ all boundaries are exercised $\Rightarrow$ a more robust test set.
- S = **Average** of the **(1 - Gini coefficient)** (over clusters)

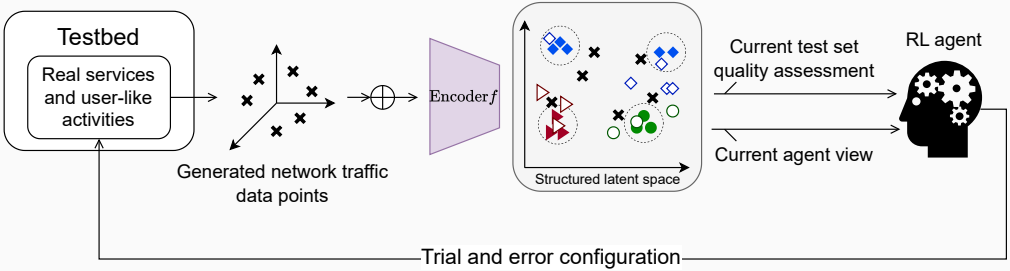


Bad scarcity



Optimal scarcity

Phase 2: Targeted Augmentation



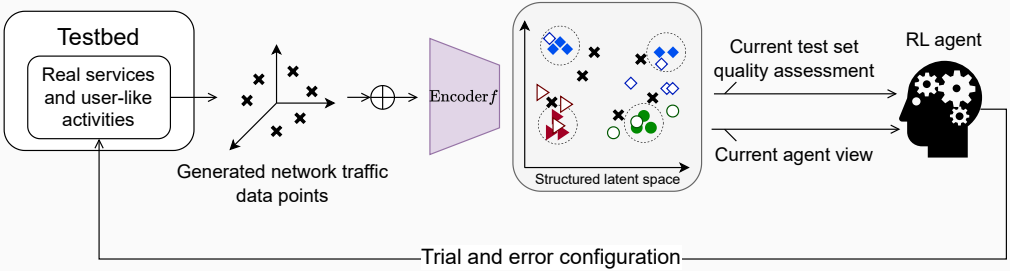
- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\boxed{\text{observe latent-space}} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$

$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation



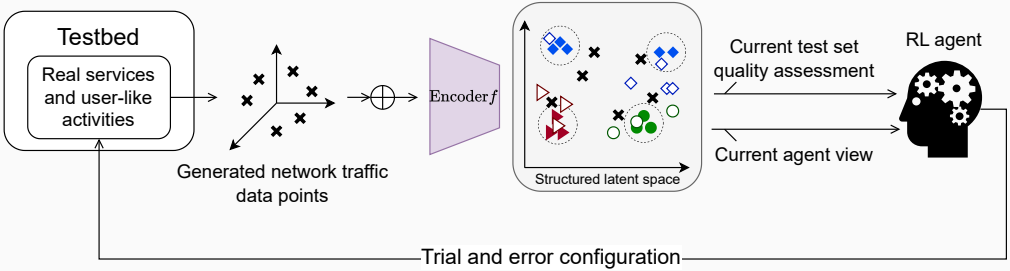
- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \boxed{\text{select a configuration}} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$

$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation



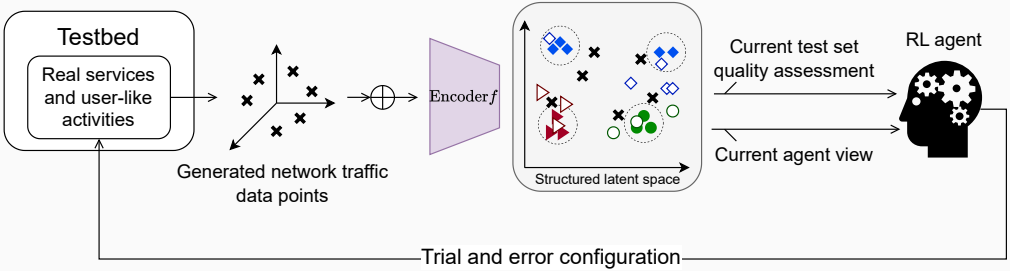
- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \boxed{\text{traffic generation}} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$

$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation



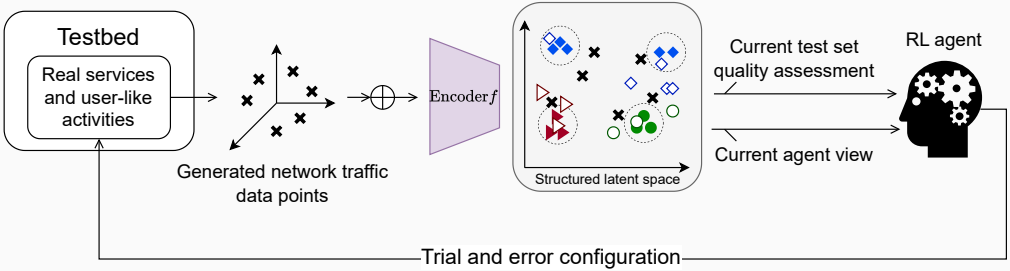
- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \boxed{\text{update } \mathcal{Z}_{\text{test}}}$$

$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation

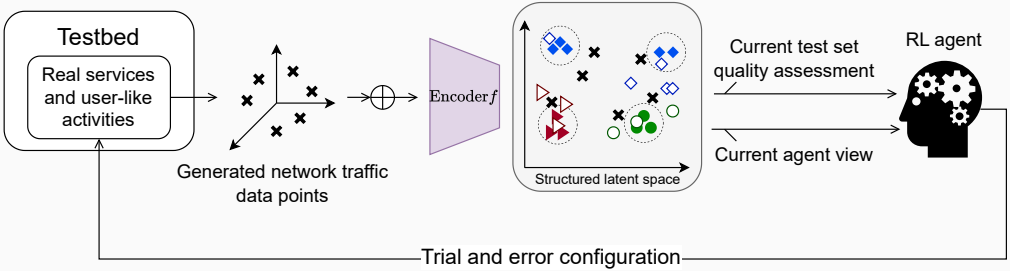


- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \boxed{\text{compute proximity, diversity, and scarcity}} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation

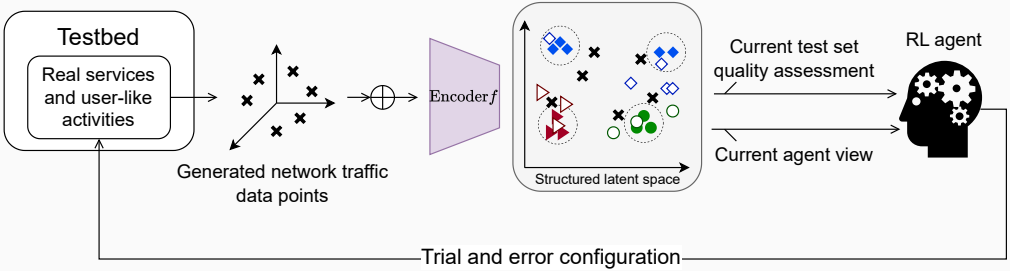


- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \boxed{\text{update policy } \pi} \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation

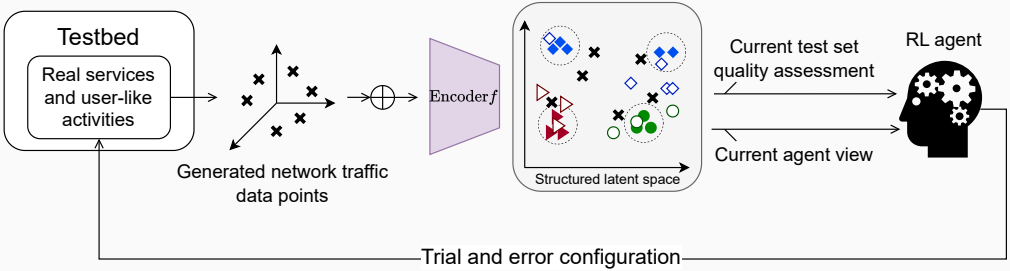


- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \; \boxed{\circlearrowright} \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation



- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \boxed{\pi^*}$$

Experimental Setup

Datasets (60 / 20 / 20 split)

(Using the revised and corrected versions)

- **CIC-IDS 2017**: 0.93 M benign / 0.25 M attack flows
- **CSE-CIC-IDS 2018**: 59.8 M benign / 4.1 M attack flows

NIDS Models

- RF, SVM, 3-layer DNN (128-128-128 ReLU + Softmax)

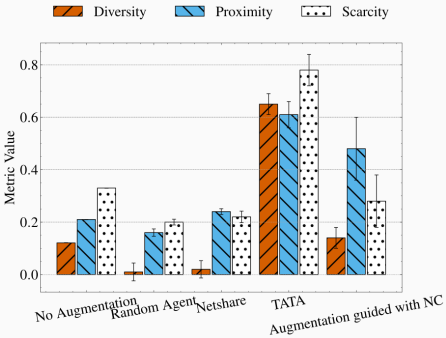
Agent Training

- Augmentation targets Benign SSH flows only

- Action space (client-side via `tc`): **Loss** $\in [5, 10]\%$ | **Jitter** $\in [4, 10]\,\mathrm{ms}$ | **Delay** $\in [10, 40]\,\mathrm{ms}$ | **Duplication** $\in [0.1, 5]\%$ | **Corruption** $\in [0.1, 10]\%$ | **Reordering** $\in [0.1, 50]\%$ | **Correlation** $\in [50, 100]\%$.

Test-set Augmentation — Metrics

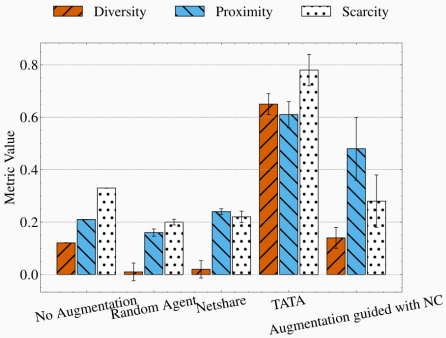
- **Protocol:** 20 eval episodes on IDS2017; no reward/updates; add Benign SSH flows; report mean±sd.
- **Baselines:** Random, NetShare (GAN), RL + Neuron Coverage (NC).



Test-set quality metrics (higher is more challenging/covered).

Test-set Augmentation — Metrics

- **Protocol:** 20 eval episodes on IDS2017; no reward/updates; add Benign SSH flows; report mean±sd.
- **Baselines:** Random, NetShare (GAN), RL + Neuron Coverage (NC).
- **Quality metrics:**
  - **TATA:** $D + \sim 437\%$; $P + \sim 190\%$; $S + \sim 136\%$
  - **NC:** $D \approx 0$; $P + \sim 129\%$; $S \approx 0$
  - **Random/NetShare:** $D \downarrow$ to ≈ 0; $P \approx 0$; $S \downarrow$

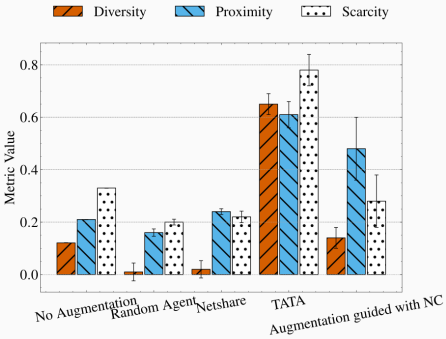


Test-set quality metrics (higher is more challenging/covered).

Test-set Augmentation — Metrics

- **Protocol:** 20 eval episodes on IDS2017; no reward/updates; add Benign SSH flows; report mean±sd.
- **Baselines:** Random, NetShare (GAN), RL + Neuron Coverage (NC).
- **Quality metrics:**
  - **TATA:** $D + \sim 437\%$; $P + \sim 190\%$; $S + \sim 136\%$
  - **NC:** $D \approx 0$; $P + \sim 129\%$; $S \approx 0$
  - **Random/NetShare:** $D \downarrow$ to ≈ 0; $P \approx 0$; $S \downarrow$

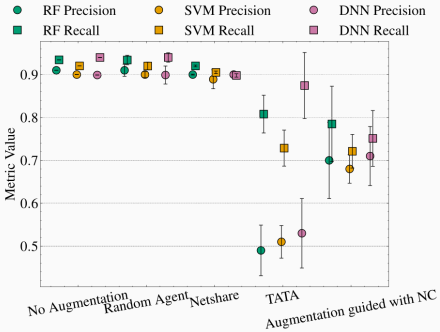
What is the impact on NIDS performance?



Test-set quality metrics (higher is more challenging/covered).

Test-set Augmentation — Impact on NIDS

- **NIDS impact (RF example):**
  - **TATA:** Precision $-\sim 46\%$; Recall $-\sim 13\%$
  - **NC:** Precision $-\sim 23\%$; Recall $-\sim 16\%$
- **Note:** higher scarcity spreads errors across boundaries $\Rightarrow$ FP $\uparrow \Rightarrow$ precision $\downarrow$.
- $\Rightarrow$ **proximity is the key driver;** scarcity and diversity are complementary (spread errors, avoid redundancy)



Impact on detection metrics (example: RF).

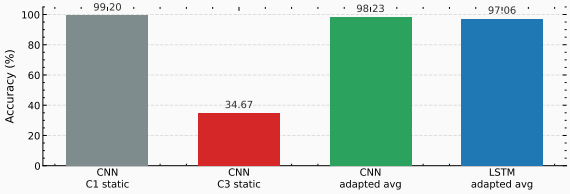
Part I conclusion

- TATA provides a pre-deployment framework to assess whether a $\mathcal{D}_{\text{test}}$ really challenges a NIDS.
- Targeted augmentation with a configurable live testbed exposes hidden weaknesses while preserving realistic traffic generation.
- Across **12 public NIDS datasets** released between **2009 and 2020**, the metrics remain mostly below **0.5**, suggesting only limited test-set difficulty.
- A more reliable pre-deployment evaluation is necessary, but not sufficient; robust ML-based NIDS also require post-deployment adaptation.

Part II: Auto-configuration

Why ML-based NIDS still need explicit adaptation

- ML-based NIDS are trained on historical traffic and deployed as static detectors.
- This assumes training and deployment traffic remain statistically close.
- Benign traffic evolves, attackers adapt → the network context changes.



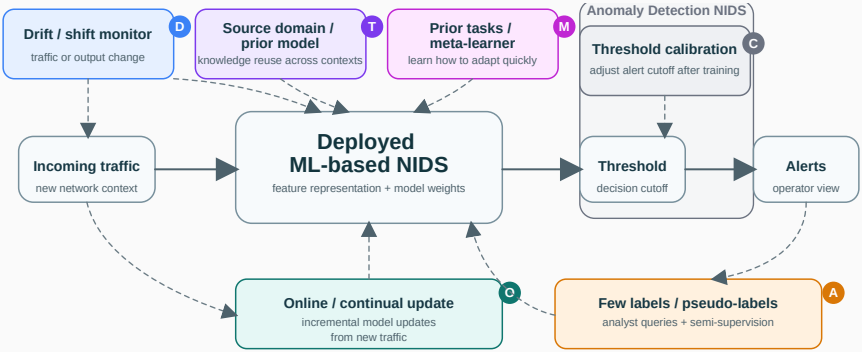
Source: **Qiao et al., IEEE TII 2022, Table V/VI and Section IV-C**

⇒ **Detection performance can degrade** ⇒ **explicit adaptation mechanisms are needed**

Adaptation = post-deployment update of some part of the detection pipeline to maintain performance under network context change.

e.g., the input data, the model, the hyperparameter configuration, or the threshold

State of the art: main adaptation families



D Drift detection
monitor change in traffic or outputs
CADE, OWAD, TRANSCEND

O Online / continual learning
update model weights over time
AOC-IDS, ENIDrift, ReCDA

A Active / semi-supervised
use few labels or pseudo-labels
Active-SVDD, HITL-IDS, METALS

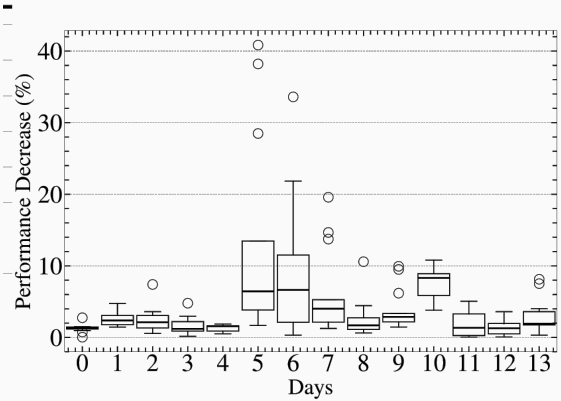
T Transfer / domain adaptation
reuse knowledge from another context
ADA, NAEF, DI-NIDS

M Meta-learning
learn how to adapt quickly from prior tasks
FC-Net, PTN-IDS, RETSINA

C Threshold calibration
adjust alert cutoff after training
Bridges, Hundman, MAGPIE

Is hyperparameter configuration important?

- We test whether a configuration optimized for one day can be reused on another day.
- Each boxplot summarizes the **performance decrease** when transferring the optimal configuration from other days.
- Some days show a limited median loss, but others reach about **20–40%** degradation.



Source: Anser et al., IEEE/IFIP NOMS 2026, Figure 5

⇒ **Therefore, configuration transfer can help in some contexts, but it is not reliable as a general strategy.**

How is configuration handled today?

Conventional offline HPO

- Grid search, random search, Bayesian optimization, and evolutionary search.
- Repeated train-and-validate evaluations over candidate configurations.

Why not for ML-based NIDS adaptation?

These methods are static, offline, computationally expensive, and time-consuming procedures that must be re-run from scratch when the network context changes.

A NIDS-specific example

In Anser et al. (NOMS 2023), Bayesian optimization of a binary Random Forest NIDS used **300 iterations** for a single daily network context, which corresponds to about **50 minutes** when one HPO iteration takes up to **10 seconds**.

Refs: Bergstra & Bengio, JMLR 2012; Snoek et al., NeurIPS 2012; Anser et al., NOMS 2023.

Research question (RQ1)

Research question

How can we find a configuration of a ML-based NIDS in a timely manner while maintaining an acceptable level of detection performance?

From our analysis of hyperparameter impact

- A NIDS configuration method has to be applied at each network context change.
- A new network context is defined by a new attack in the network or by a topology change.

Source: Anser et al., *An Empirical Study on Configuration Robustness of Unsupervised IDS*, IEEE/IFIP NOMS 2026.

$\Rightarrow$ **Auto-configuration via meta-learning**

Auto-configuration via Meta-learning

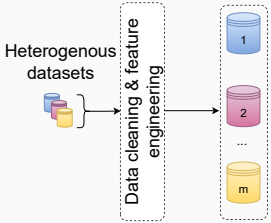
A. Meta-modeling

B. On the fly NIDS configuration

Trained NIDS

Auto-configuration via Meta-learning

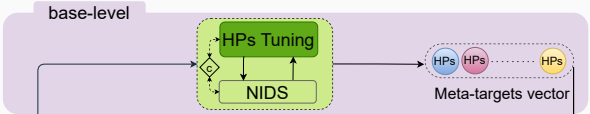
A. Meta-modeling

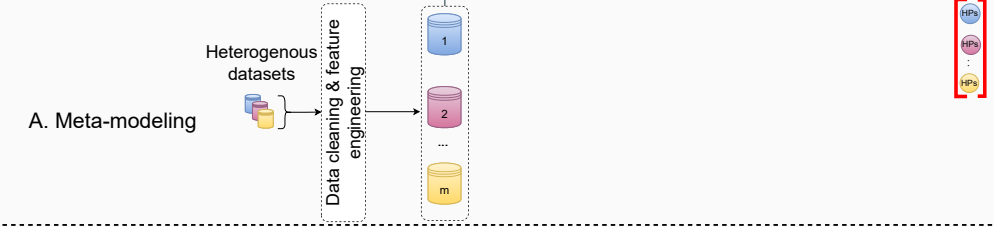


B. On the fly NIDS configuration

Auto-configuration via Meta-learning

For AD NIDS, tuning covers both HPs and threshold
HPs here = model HPs + threshold





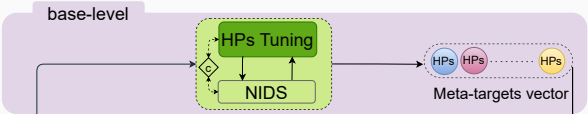
A. Meta-modeling

B. On the fly NIDS configuration

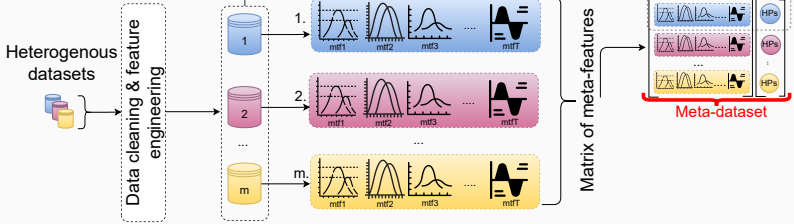
Trained NIDS

Auto-configuration via Meta-learning

For AD NIDS, tuning covers both HPs and threshold
HPs here = model HPs + threshold



A. Meta-modeling

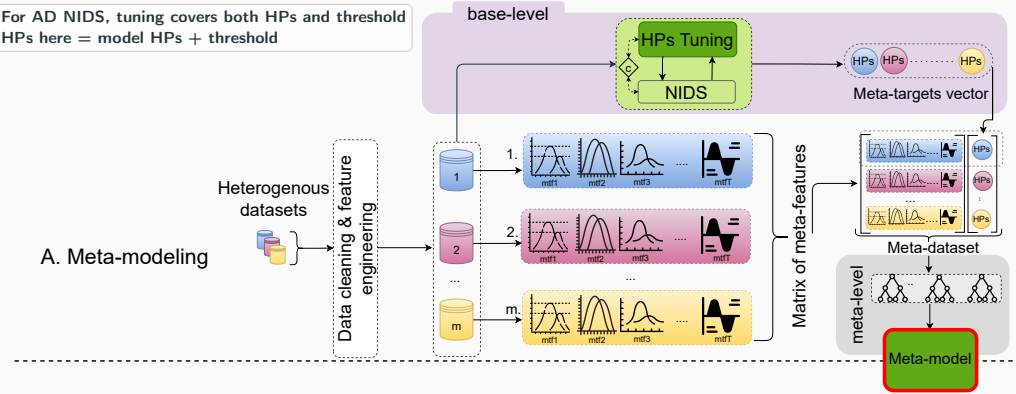


B. On the fly NIDS configuration

Trained NIDS

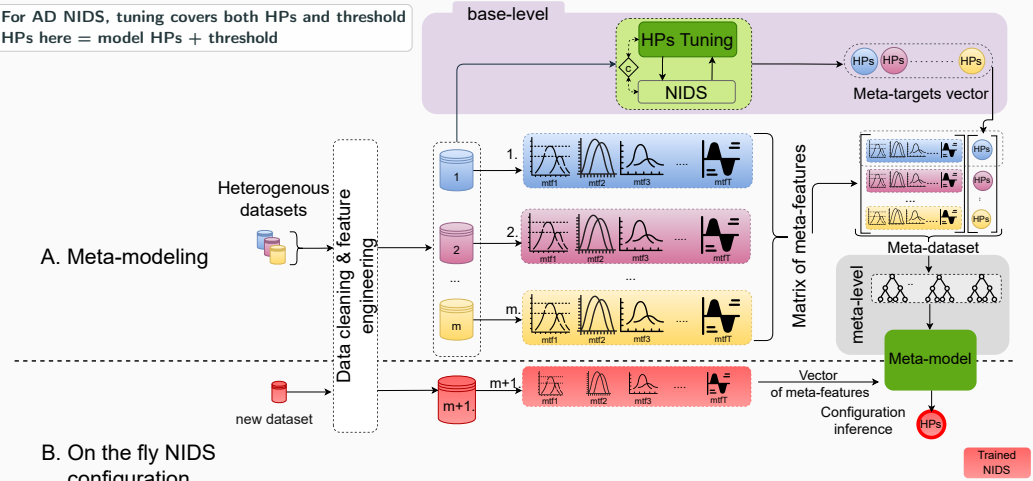
Auto-configuration via Meta-learning

For AD NIDS, tuning covers both HPs and threshold
HPs here = model HPs + threshold



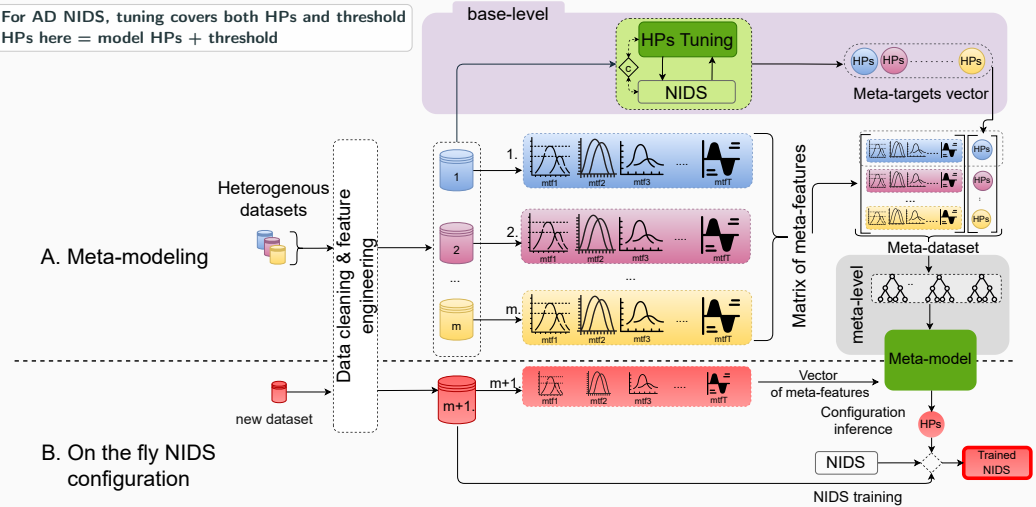
Auto-configuration via Meta-learning

For AD NIDS, tuning covers both HPs and threshold
HPs here = model HPs + threshold



Auto-configuration via Meta-learning

For AD NIDS, tuning covers both HPs and threshold
HPs here = model HPs + threshold



Methodology applied in two settings

Supervised setting

- Binary Random Forest on labeled benign/attack traffic.
- Reference: Anser et al., **NOMS 2023** (AnNet workshop).
- Contexts: **17 daily** IDS2017/IDS2018 datasets.
- Search space: **2 RF HPs** (‘max_features‘, ‘min_samples_leaf‘).
- BO baseline: **TPE**, **300 iterations/day**, optimized on **MCC**.

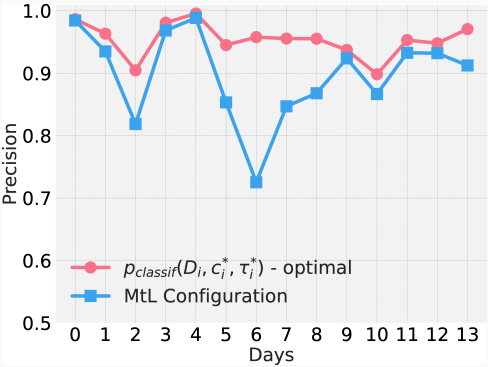
Unsupervised setting

- Isolation Forest-based AD IDS trained on attack-free data.
- Reference: Anser et al., **LCN 2024** (short paper).
- Contexts: **14 daily** IDS2017/IDS2018 datasets.
- Search space: **7 iForest HPs + threshold**.
- BO baseline: offline daily optimum with labels; metrics: **ROC-AUC**, **F1**, **precision**.

$\Rightarrow$ **Same meta-learning pipeline, but different detectors, search spaces, and targets.**

Results: precision

- In the attack-free AD setting, precision stays high overall.
- Variability is larger on days **2** and **5–8**.
- The gap comes from a few harder contexts where transfer is less reliable.
- In the supervised setting, precision is near-saturated and MtL often overlaps HPO.

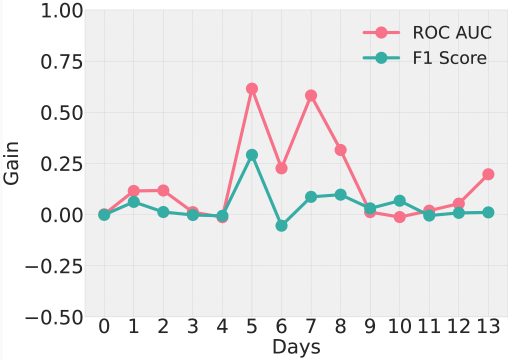


Unsupervised setting

⇒ **High precision in both settings; unsupervised AD exposes the harder transfer cases.**

Results: gain

- In the attack-free AD setting, MtL can exceed BO when BO is budget-matched.
- Gains are strongest around the harder days, especially **5–8**.
- Negative gains stay limited at equal cost.
- In the supervised setting, MtL also improves over the default with large gains on difficult days.

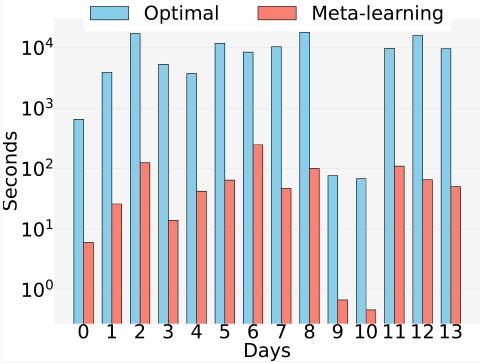


Unsupervised setting

⇒ **Transfer is cheaper than BO and can still win under the same budget.**

Results: configuration time

- In the attack-free AD setting, MtL returns a configuration in seconds to minutes.
- BO can grow by orders of magnitude and exceed **100,000 s**.
- Overall, MtL is about **160x faster**.
- In the supervised setting, MtL remains quasi-constant and is about **9x faster** than HPO on average.



Unsupervised setting

⇒ **For online reconfiguration, the time advantage becomes decisive in the attack-free AD setting.**

Auto-configuration: conclusion

- Re-running HPO after each context change is too expensive for adaptive ML-based NIDS.
- Meta-learning preserves strong detection performance while staying close to HPO in both settings.
- The operational benefit is time: visible in supervised NIDS and decisive in attack-free AD.
- For AD NIDS, auto-configuration must remain context-aware, since both model HPs and threshold must be adapted.

$\Rightarrow$ **Meta-learning makes NIDS reconfiguration fast enough for practice without sacrificing too much detection performance.**

General conclusion

- Robust ML-based NIDS should not be judged only by their accuracy on a static benchmark.
- This thesis addressed robustness along two complementary axes: **configuration robustness** and **evaluation robustness**.
- Meta-learning enables fast, context-aware reconfiguration, including attack-free AD where both model HPs and threshold must be adapted.
- TATA strengthens pre-deployment evaluation by assessing test-set difficulty and generating targeted, protocol-faithful traffic to expose hidden weaknesses.

$\Rightarrow$ **Robust ML-based NIDS require both adaptive configuration and stronger pre-deployment evaluation.**

Perspectives

- Move from one-shot benchmarking to **continuous, closed-loop evaluation and adaptation**.
- Extend both frameworks beyond the current flow-based and single-protocol settings toward richer traffic granularities, applications, and larger testbeds.
- Reuse TATA-generated hard cases not only for assessment, but also for **hardening and retraining** the detector.
- Use **GenAI** for scenario-to-traffic generation and analyst assistance, while accounting for more adaptive attackers.
- Explore **privacy-preserving and federated configuration learning** to mitigate cold-start deployment and share useful experience across organizations.

$\Rightarrow$ **The long-term goal is continuously evaluable, configurable, and trustworthy ML-based NIDS.**

Questions?